

QMP26 Case Study

PCA Factor Models & Phantom Lattice Trading | 2026

General Instructions for Students

1. This programming assignment focuses on understanding financial data in two complementary contexts: static factor model estimation and real-time market microstructure anomaly detection.
2. Submissions must be compiled into a single zip file with the filename format: Team_{TeamNumber}.zip containing the the two zip folders named {Problem_Statement_<#Number>.zip}.
3. If you encounter difficulties copying cells from provided materials, you may create your own markdown cells that closely resemble those in the original specifications.
4. Ensure a thorough understanding of your solutions, as you will be required to explain your approach and observations during case study presentations.

Case Study Structure

The case study consists of **two integrated problem statements (10 parts, 87 marks maximum)**:

- **Problem Statement 1: PCA Factor Models in Finance** (6 parts, 40 marks) — Understanding eigendecomposition, dimensionality reduction, factor models, and portfolio construction
 - **Problem Statement 2: The Phantom Lattice** (4 parts + optional bonus, 47 marks) — Real-time anomaly detection, regime classification, and risk management
-

Problem Statement 1: Understanding PCA Factor Models in Finance

Total Marks: 40

- This problem statement focuses on understanding Principal Component Analysis applied to financial data, covering eigendecomposition, dimensionality reduction, factor models, and

portfolio construction.

- You will work with synthetic stock price data generated from a factor model and analyze its statistical properties through PCA.
- Each of the 6 parts builds upon the previous concepts: from basic data properties to advanced portfolio construction and model stability.
- Use the provided code skeletons as starting points, but feel free to implement your own approaches.

Quantitative Methods in Practice | 2026 | Total Marks: 40

Problem-Specific Instructions

This programming assignment focuses on understanding Principal Component Analysis (PCA) as applied to financial data, covering eigendecomposition, dimensionality reduction, factor models, and portfolio construction. Your notebook must be able to execute completely using the "Run All" command. Any runtime errors in the submitted notebook will result in a score of zero. Ensure all plots are properly labeled. Use supporting plots to explain your results wherever necessary. Report your observations in a markdown cell below the code for each section.

Plotting data and intermediate results is highly encouraged for debugging. However, remove any debugging or exploratory code before submission. Submit your notebook with the filename format: `Team_{TeamNumber}.ipynb`. The notebook must run seamlessly; otherwise, it will not be evaluated. Submissions must be compiled into a single zip file containing the .ipynb notebook file.

Ensure a thorough understanding of your solutions, as you will be required to explain your approach and observations during your case study presentations. You do not need to submit the PowerPoint presentation, only the notebook. The case study is divided into six parts, each following the marking scheme below:

Part	Topic	Marks
1	Understanding Returns and Covariance Structure	5
2	Understanding PCA and Eigendecomposition	5
3	Understanding Reconstruction and Dimensionality Reduction	5
4	Understanding Information Leakage in Unsupervised Learning	5
5	Understanding Factor-Neutral Portfolio Construction	10
6	Understanding Stability, Robustness and Rolling PCA	10

Part 1: Understanding Returns and Covariance Structure

5 marks

Assume you are given daily closing prices of 10 synthetic stocks over 500 trading days.

The data-generating process is:

$$P_{i,t} = P_{i,t-1} \cdot \exp(\mu_i + \beta_i f_t + \varepsilon_{i,t})$$

where $f_t \sim N(0, 0.01)$ is a common market factor, $\beta_i \in [0.5, 1.5]$ are factor loadings, μ_i are small drifts, and $\varepsilon_{i,t} \sim N(0, \sigma_i^2)$ is idiosyncratic noise.

(a) Generate 500 days of prices for 10 stocks using the process above. Choose reasonable parameters: $P_{i,0} = 100$, $\mu_i \sim U(0.0001, 0.0005)$, $\beta_i \sim U(0.5, 1.5)$, $\sigma_i \sim U(0.005, 0.02)$. Plot all 10 price series on a single chart.

(b) Compute daily log returns: $r_{i,t} = \ln(P_{i,t} / P_{i,t-1})$. Explain in a markdown cell why PCA must be performed on returns (or a stationary transformation) rather than on prices directly.

(c) Compute the 10×10 correlation matrix and the 10×10 covariance matrix of the returns. Visualize both as heatmaps. Identify which pair of stocks has the highest and lowest correlation.

(d) Standardize the returns to zero mean and unit variance. Explain why standardization matters when stocks have different volatilities. What happens to PCA results if you skip this step?

(e) Write down all your observations.

```
# Part 1: Understanding Returns and Covariance Structure
# All imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

#####
# (a) Generate synthetic stock prices
#####
N_stocks = 10
N_days = 500

# Set parameters
P0 = 100 # Initial price for all stocks
mu = np.random.uniform(0.0001, 0.0005, N_stocks) # Drift per stock
```

```

beta = np.random.uniform(0.5, 1.5, N_stocks)          # Factor loading per stock
sigma = np.random.uniform(0.005, 0.02, N_stocks)      # Idiosyncratic vol per stock

# Generate common factor
# f_t ~ N(0, 0.01)

# Generate prices using the DGP
# P_i,t = P_i,t-1 * exp(mu_i + beta_i * f_t + eps_i,t)

# Plot all 10 price series

#####
# (b) Compute log returns
#####
def compute_log_returns(prices):
    # Compute log returns from price matrix
    # returns shape: (N_days-1, N_stocks)
    return log_returns

# Explain in markdown: why returns not prices?

#####
# (c) Correlation and Covariance matrices
#####
# Compute correlation matrix
# Compute covariance matrix
# Plot heatmaps (use seaborn)
# Identify max and min correlation pairs

#####
# (d) Standardize returns
#####
def standardize_returns(returns):
    # Zero mean, unit variance per stock
    return standardized_returns, scaler

# Explain: what happens to PCA if you skip standardization?

```

Report your observations:

- 1.
- 2.
- 3.

Part 2: Understanding PCA and Eigendecomposition

5 marks

- (a) Perform PCA on the standardized returns from Part 1(d). Extract eigenvalues and eigenvectors. Plot a scree chart showing the proportion of variance explained by each component. How many components are needed to explain $\geq 80\%$ of total variance?
- (b) Now perform PCA on the covariance matrix (i.e., on unstandardized returns). Compare the eigenvalues and explained variance ratios with those from (a). Are they the same? Explain why or why not, with reference to the differing stock volatilities.
- (c) Inspect the loadings (eigenvector components) of PC1. Are all loadings the same sign? What does this represent financially? Now inspect PC2 — do the loadings split stocks into groups? Hypothesize what "factor" PC2 could represent.
- (d) Compute the scores (projections of each day's returns onto the principal components). Plot the PC1 score time series. Does it resemble any known market quantity?
- (e) A colleague claims: "The loadings vector IS the same thing as the scores vector." Disprove this claim by showing their shapes, units, and interpretations differ.
- (f) Report all your observations.

```
# Part 2: Understanding PCA and Eigendecomposition

from sklearn.decomposition import PCA

#####
# (a) PCA on standardized returns (correlation PCA)
#####
def perform_pca(data, n_components=None):
    # Fit PCA
    # Return: pca object, eigenvalues, eigenvectors, explained_variance_ratio
    return pca, eigenvalues, eigenvectors, explained_var_ratio

# Fit PCA on standardized returns
# Plot scree chart: explained variance ratio vs component number
# Determine number of components for  $\geq 80\%$  variance

#####
# (b) PCA on covariance matrix (unstandardized returns)
#####
# Fit PCA on raw (unstandardized) returns
# Compare eigenvalues and explained variance ratios
```

```

# Plot both scree charts side by side

#####
# (c) Interpret loadings
#####
def plot_loadings(eigenvectors, component_idx, stock_names):
    # Bar plot of loadings for a given PC
    pass

# Plot loadings for PC1 and PC2
# Discuss sign patterns

#####
# (d) Compute and plot scores
#####
def compute_scores(standardized_returns, eigenvectors):
    # Project returns onto PCs
    # scores = X @ V (where V is eigenvector matrix)
    return scores

# Plot PC1 scores over time

#####
# (e) Disprove "loadings = scores"
#####
# Print shapes of loadings vs scores
# Explain difference: loadings are (N_stocks,) per PC; scores are (N_days,) per PC

```

Report your observations:

- 1.
- 2.
- 3.

Part 3: Understanding Reconstruction and Dimensionality Reduction

5 marks

(a) Using only the first $k = 1, 2, 3$ principal components, reconstruct the original standardized returns matrix. The reconstruction formula is:

$$\hat{X}_k = \text{Scores[:, :k]} \times \text{Loadings[:k, :]}$$

Compute the mean squared reconstruction error (MSRE) for each value of k . Plot MSRE vs k .

(b) For each stock, compute the per-stock reconstruction error using $k=3$ components. Which stock has the highest residual error? What does this mean financially? (Hint: it is the stock least explained by common factors — most "idiosyncratic.")

(c) Reconstruct the returns using $k = 10$ (all components). Verify that the reconstruction error is zero (within floating-point tolerance). Explain why this must be true mathematically.

(d) A data vendor provides a "compressed returns file" where they stored only PC1–PC3 scores and loadings. You reconstruct returns from this. Compare the correlation structure of the reconstructed returns with the original. Plot both correlation heatmaps side by side. What is lost?

(e) Report all your observations.

```
# Part 3: Understanding Reconstruction and Dimensionality Reduction
```

```
#####
```

```
# (a) Reconstruct returns using k components
```

```
#####
```

```
def reconstruct(scores, loadings, k):
```

```
    #  $X_{\text{hat}} = \text{scores[:, :k]} @ \text{loadings[:, :k]}$ 
```

```
    return  $X_{\text{hat}}$ 
```

```
def compute_msre( $X_{\text{original}}$ ,  $X_{\text{reconstructed}}$ ):
```

```
    # Mean squared reconstruction error
```

```
    return msre
```

```
# Iterate  $k = 1, 2, \dots, 10$ 
```

```
# Compute MSRE for each  $k$ 
```

```
# Plot MSRE vs  $k$ 
```

```
#####
```

```
# (b) Per-stock reconstruction error ( $k=3$ )
```

```
#####
```

```
# Compute error per column (stock)
```

```
# Identify the most idiosyncratic stock
```

```
# Explain financial meaning
```

```
#####
```

```
# (c) Full reconstruction ( $k=10$ )
```

```
#####
```

```
# Verify error  $\approx 0$ 
```

```
# Explain: orthonormal basis spans full space
```

```
#####
# (d) Correlation structure comparison
#####
# Reconstruct with k=3
# Compute correlation of reconstructed returns
# Plot side-by-side heatmaps: original corr vs reconstructed corr
# Discuss: what information is lost in compression?
```

Report your observations:

- 1.
- 2.
- 3.

Part 4: Understanding Information Leakage in Unsupervised Learning

5 marks

- (a) Split the data temporally: Train = first 350 days, Test = last 149 days. Fit PCA on the training set only. Record the eigenvectors and the explained variance ratio on the training set.
- (b) Project the test returns onto the training eigenvectors. Compute the explained variance ratio on the test set. Compare with the training set ratio. Is there a drop? Discuss why.
- (c) Now fit PCA on the entire dataset (train + test combined). Report the explained variance. Then project only the test portion onto these full-data eigenvectors and report the test explained variance. Compare this number with (b). Explain why this procedure is invalid (information leakage) even though "PCA is unsupervised."
- (d) Quantify the leakage: compute the difference in test-set explained variance between the correct approach (b) and the leaky approach (c). Under what conditions would this gap be largest?
- (e) Report all your observations.

```
# Part 4: Understanding Information Leakage in Unsupervised Learning

#####
# (a) Temporal train/test split + PCA on train only
#####
train_returns = standardized_returns[:350, :]
test_returns = standardized_returns[350:, :]
```



```

# Fit PCA on train only
# Record eigenvectors (loadings) and explained variance ratio on train

#####
# (b) Project test data onto TRAIN eigenvectors
#####
def compute_explained_variance_oos(test_data, train_loadings, k):
    # Project test onto first k train PCs
    # Compute variance explained = 1 - (residual variance / total variance)
    return explained_var_ratio_test

# Compare train vs test explained variance for k = 1, 2, 3

#####
# (c) LEAKY approach: PCA on full data
#####
# Fit PCA on ALL data (train + test)
# Report explained variance on full data
# Project only test portion onto full-data eigenvectors
# Report test explained variance
# Explain why this is INVALID

#####
# (d) Quantify the leakage gap
#####
# Difference in test explained variance: leaky - correct
# Discuss: when is leakage worst? (e.g., regime changes, short datasets)

```

Report your observations:

- 1.
- 2.
- 3.

Part 5: Understanding Factor-Neutral Portfolio Construction

10 marks

(a) Using the PCA fitted on training data (Part 4a), compute the PC1 exposure (factor loading) for an equal-weight long portfolio of stocks 1–5. The exposure is:

$$exposure_{PC1} = w^T \cdot v_1$$

where w is the portfolio weight vector and v_1 is the PC1 loading vector.

(b) Construct a factor-neutral portfolio: find a set of portfolio weights such that (i) the portfolio is dollar-neutral (weights sum to zero), (ii) PC1 exposure is zero, and (iii) the portfolio has some desired characteristic (e.g., long the top-5 highest-drift stocks, short the rest). Formulate this as a constrained linear system and solve.

(c) Backtest both portfolios (equal-weight long vs factor-neutral) on the test period (last 149 days). Compute and compare: cumulative return, annualized volatility, Sharpe ratio, and maximum drawdown.

(d) Now construct a portfolio neutral to both PC1 and PC2. Does the additional constraint always reduce risk? Compare volatility of PC1-only-neutral vs PC1+PC2-neutral. Discuss when additional hedging helps vs when it is counterproductive.

(e) Add a transaction cost model: each rebalance costs 5 bps per unit of turnover. If the factor-neutral portfolio requires monthly rebalancing (because loadings drift), compute the break-even: at what turnover level does the hedging benefit disappear?

(f) Report all your observations.

```
# Part 5: Factor-Neutral Portfolio Construction

#####
# (a) Compute PC1 exposure of equal-weight portfolio
#####
def compute_factor_exposure(weights, loadings, pc_idx):
    # exposure = weights @ loadings[pc_idx]
    return exposure

# Equal weight long on stocks 0-4
w_long = np.array([0.2, 0.2, 0.2, 0.2, 0.2, 0, 0, 0, 0])
# Compute PC1 exposure

#####
# (b) Construct factor-neutral portfolio
#####
def construct_neutral_portfolio(target_weights, loadings, neutral_pcs=[0]):
    """
    Find portfolio weights w such that:
    1. w @ loadings[pc] = 0 for each pc in neutral_pcs (factor neutral)
    2. sum(w) = 0 (dollar neutral)
    3. Minimize ||w - target_weights||^2 (stay close to desired portfolio)
```

```

Hint: use constrained least squares or Lagrange multipliers
"""

# Set up constraint matrix A and vector b
#  $A @ w = b$ 
# Solve using scipy.optimize or analytical Lagrangian
return w_neutral

# Solve for PC1-neutral weights

#####
# (c) Backtest on test period
#####
def backtest(weights, returns):
    # Daily portfolio returns = returns @ weights
    # Compute: cumulative return, annualized vol, Sharpe, max drawdown
    return metrics_dict

# Compare equal-weight vs factor-neutral

#####
# (d) Neutral to PC1 + PC2
#####
# Construct double-neutral portfolio
# Compare volatility: single-neutral vs double-neutral

#####
# (e) Transaction cost analysis
#####
def compute_turnover(weights_old, weights_new):
    return np.sum(np.abs(weights_new - weights_old)) / 2

# Assume monthly rebalance with 5bps cost per unit turnover
# Compute net-of-cost Sharpe
# Find break-even turnover

```

Report your observations:

- 1.
- 2.
- 3.

Part 6: Understanding Stability, Robustness and Rolling PCA

10 marks

(a) Divide the full dataset into two equal halves (days 1–250 and days 251–500). Fit PCA separately on each half. Compare the PC1 loading vectors using cosine similarity:

$$\cos(\theta) = (v_{1,first}^T \cdot v_{1,second}) / (||v_{1,first}|| \cdot ||v_{1,second}||)$$

Is PC1 stable across time? What about PC2 and PC3?

(b) Implement a rolling PCA: use a rolling window of 100 days, stepping by 20 days. At each step, fit PCA and record the PC1 loadings. Plot how each stock's PC1 loading evolves over time. Discuss what causes loading instability.

(c) Sign-flip problem: eigenvectors are defined only up to a sign (if v is an eigenvector, so is $-v$). Implement a sign-correction algorithm: at each rolling window, flip the sign of the new eigenvector if its cosine similarity with the previous window's eigenvector is negative. Show before vs after plots.

(d) Generate a regime-change scenario: for days 1–250 use β_i values from Part 1; for days 251–500, swap the β values of stocks 1–5 and stocks 6–10 (simulating a sector rotation). Re-run rolling PCA. Show that PC1 loadings now shift dramatically at the regime boundary. Discuss implications for a factor-neutral portfolio that doesn't re-estimate.

(e) Using your rolling PCA from (b), implement a walk-forward factor-neutral strategy: at each rebalance date, use only past-window eigenvectors to construct the hedge. Backtest this strategy and compare its OOS performance to the "oracle" strategy (which uses future eigenvectors). Quantify the performance gap.

(f) Report all your observations. Summarize: under what conditions is PCA a reliable tool for factor modeling, and when does it break down?

```
# Part 6: Stability, Robustness and Rolling PCA

#####
# (a) Split-half stability test
#####
def cosine_similarity(v1, v2):
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))

# Fit PCA on first half and second half
# Compare PC1, PC2, PC3 loadings via cosine similarity
```

```
#####
# (b) Rolling PCA
#####
def rolling_pca(returns, window=100, step=20):
    """
    Fit PCA on each rolling window. Return: list of (start_idx, eigenvalues, loadings) t
    """
    results = []
    for start in range(0, len(returns) - window, step):
        window_data = returns[start:start+window]
        # Standardize within window
        # Fit PCA
        # Store results
    return results

# Plot PC1 loadings over time (one line per stock)

#####
# (c) Sign-flip correction
#####
def correct_sign_flips(loadings_over_time):
    """
    For each time step  $t > 0$ :
    If  $\text{cosine\_similarity}(\text{loadings}[t], \text{loadings}[t-1]) < 0$ :
        flip  $\text{loadings}[t] *= -1$ 
    """
    corrected = [loadings_over_time[0]]
    for t in range(1, len(loadings_over_time)):
        if cosine_similarity(loadings_over_time[t], corrected[t-1]) < 0:
            corrected.append(-loadings_over_time[t])
        else:
            corrected.append(loadings_over_time[t])
    return corrected

# Plot before vs after correction

#####
# (d) Regime change scenario
#####
# Generate new data with swapped betas at day 250
# Run rolling PCA
# Show loading shift at regime boundary
# Discuss: what happens to a static factor-neutral portfolio?
```

```
#####
# (e) Walk-forward factor-neutral backtest
#####
def walk_forward_backtest(returns, window=100, rebalance_freq=20):
    """
    At each rebalance date:
    1. Fit PCA on past 'window' days only
    2. Construct factor-neutral weights using those loadings
    3. Hold for 'rebalance_freq' days
    4. Record daily P&L
    """
    # Compare to oracle strategy (uses future eigenvectors)
    return walk_forward_metrics, oracle_metrics

# Quantify performance gap
```

Report your observations:

- 1.
- 2.
- 3.

QMP26 Case Study | PCA Factor Models in Finance | 2026

Problem Statement 2: The Phantom Lattice — Market Microstructure & Real-Time Risk

Total Marks: 47 (including optional +5 bonus)

- This problem statement focuses on discovering hidden factor structure in a multi-asset trading book using unsupervised learning, building real-time anomaly detectors, and defending operational decisions with quantitative evidence.
- You are given tick data from a multi-asset market-making book and must discover its latent factor structure, detect anomalies, classify market regimes, and model inventory dynamics.
- Unlike Problem Statement 1 (static analysis), this problem requires streaming algorithms and real-time decision-making.
- The bonus section rewards teams who integrate all four parts into a single end-to-end pipeline.

You have recently joined a desk that runs a single market-making book in 12 correlated synthetic assets. You are given four days of tick data (ticks_mon.csv through ticks_thu.csv) and a kill-

switch alert from Tuesday afternoon. There is no model documentation and no factor file. Your task, in four parts, is to discover the latent factor structure of the book, build a real-time anomaly detector, classify the market regime tick by tick, and quantify the probability that the inventory limits are breached before the per-session round-trip target is met.

Difficulty note. Parts 1(c) and 2 use concepts (the Q-statistic, Hotelling's T^2 , and streaming PCA) that may not have been covered in your coursework. A short primer is provided in **Appendix A**; please read it before starting Part 2. The case study is pitched above the standard undergraduate level, and candidates are expected to read and experiment as they work through it.

Data Pack and Conventions

Files shipped to you (in the case-study zip):

- `ticks_mon.csv`, `ticks_tue.csv`, `ticks_wed.csv`, `ticks_thu.csv` – long-form tick data with columns `timestamp` (timezone-aware `datetime64[ns, UTC]`), `asset_id` (string in S01 ... S12), and `mid_return` (float, log-return between consecutive mid-prices). Each day contains on the order of 25–35 thousand surviving ticks per asset after the any-asset-NaN drop in Part 1.
- `labels_wed.csv` – ground-truth Wednesday regime labels with columns `timestamp` and `regime` (one of NORMAL, FAT_FINGER, SECTOR_ROTATION, CORRELATION_BREAK). Used for self-scoring in Part 3(c) and may be used as training data for Part 3(b).
- `score_submission` code – **the Wednesday self-scoring tool. We have pasted the code in Appendix B below.**

Files held back by the panel:

- `labels_thu.csv` – the hidden Thursday ground truth used to score Part 3(d).

Conventions that apply across all parts:

- The 13:00 cut-off in Part 2 and any other wall-clock references are evaluated against the UTC `timestamp` column.
- The number of retained principal components, K (or k in Part 2), is chosen *once* in Part 1(a) and re-used unchanged in Parts 2, 3, 4 and the Bonus. Do not re-pick K per part.
- Any randomness in your code (Monte Carlo, Isolation Forest, KMeans, train/test splits, etc.) must be explicitly seeded so that “Run All” is reproducible. Non-deterministic but error-free output is acceptable provided seeds are set; the runtime-error rule in the General Instructions is about exceptions, not about exact bit-for-bit reproducibility.

Parts and Marking Scheme (4 Parts + Bonus, 47 marks total)

- **Part 1. The Blind Cartographer.** 10 marks Discover the latent factor structure of the book using PCA. Interpret what you find, demonstrate an understanding of information leakage in unsupervised learning, and prove portfolio neutrality from the geometry of the loadings.
 - **Part 2. The Sentinel.** 10 marks Build a PCA-based anomaly detection system for Tuesday's tick stream. Implement the Q and T² statistics, set thresholds, perform contribution analysis, build a streaming detector, and benchmark against at least two alternative methods.
 - **Part 3. The Regime Hunt.** 10 marks An open-ended exercise. Wednesday's and Thursday's data contain multiple regime shifts. Build the best streaming regime classifier you can; submissions are scored on a held-out test set.
 - **Part 4. The Kill-Switch.** 10 marks Model the market-making engine's inventory as an asymmetric random walk between two absorbing barriers. Derive the closed-form ruin probability, study its sensitivity to the latency-induced asymmetry, and use your Part 2 detector to drive dynamic barriers.
 - **Bonus. The Full Stack.** +5 marks OPTIONAL Wire Parts 1 to 4 into a single end-to-end pipeline and run it on a simulated one-hour feed.
-

Part 1 • The Blind Cartographer

The book holds an **equal-weight** exposure across 12 synthetic assets labelled S01 ... S12. The PM claims the book is *market-neutral by construction*: a single dollar of any macro factor moving against the desk should produce *exactly* zero P&L impact, not "approximately" zero, and not "small on average". The factor file, however, has been lost. There is no `factors.csv`; there is only tick data.

Your job is to **discover the factors yourself** using unsupervised learning, and then prove the neutrality claim from the structure you uncover.

(a) Discover the latent factor structure 2 marks

Load `ticks_mon.csv` (columns `timestamp`, `asset_id`, `mid_return`) and pivot it into a $(T \times 12)$ matrix of returns aligned on timestamps. The 12 assets share a common timestamp grid on each day; any timestamp with a missing value for any asset should be dropped. Fit PCA on the **centred** returns (no scaling) using the **covariance** matrix; do not standardise the columns. `sklearn.decomposition.PCA` with default settings does exactly this.

- Produce a **scree plot** of the eigenvalues.
- Determine the number of meaningful principal components using **at least two** of the following criteria: the Kaiser criterion (here, eigenvalue greater than the mean eigenvalue, since we are working with a covariance matrix on unstandardised returns); the broken-stick test (retain component j if its eigenvalue exceeds $\lambda_j^{bs} = \bar{\lambda} \sum_{i=j}^{12} 1/i$, where $\bar{\lambda}$ is the mean

eigenvalue); parallel analysis (Horn's method, using 500 resamples obtained by independently shuffling each column of the centred Monday returns to break cross-asset correlation while preserving marginals, and retaining components whose eigenvalue exceeds the 95th percentile of the corresponding resampled eigenvalue); or the explained-variance-elbow rule. State each criterion's verdict separately. If the criteria disagree, state which single value of K you adopt for Parts 2 to 4 and the Bonus, and justify the choice in one sentence; this K is then used unchanged everywhere downstream.

- Report the eigenvalues and explained-variance ratios of the top components.

(b) Interpret the structure 2 marks

- Plot the 12 asset loadings as points in the (PC1, PC2) plane. By "first two PCs" we mean the two components with the largest eigenvalues. Annotate each point with its asset id.
- Note that PCA eigenvectors are defined only up to a sign. The geometric shape of the loadings cloud is invariant to such sign flips and to mirror reflections across an axis; do not worry if your picture is rotated or reflected relative to a neighbouring team's.
- Describe the geometric arrangement in plain English. Is there a symmetry? Are the points clustered, spread evenly, or collinear? You should expect the arrangement to hold only approximately, up to sampling noise. (Hint: consider whether the 12 points resemble the vertices of a regular polygon, i.e. the roots of unity in the complex plane, $e^{i \cdot 2\pi k / 12}$ for $k = 0, \dots, 11$.)
- Asset ids in this dataset are arbitrary labels with no real-world counterparts. You are not expected to map PC1 and PC2 to specific named macro factors; instead, describe the *type* of structure each PC encodes (e.g. a level / co-movement factor, a dispersion factor, a sector tilt) and justify your description using the signs and magnitudes of the loadings.
- Now rotate your top-2 loadings *in the (PC1, PC2) plane* by an arbitrary angle (say 30°) and re-plot. Project Monday's centred returns onto the two rotated axes to obtain a pair of rotated score series, and compute the variance of each series; the **sum** of these two variances is the "jointly explained variance". Compare it to the sum of the variances of the un-rotated PC1 and PC2 scores (i.e. $\lambda_1 + \lambda_2$). Has the per-axis decomposition changed? Has the joint figure changed? Has your interpretation of "PC1" and "PC2" changed? Discuss what this tells you about the **uniqueness** of PCA factors versus "true" factors. (Hint: this is the rotation-ambiguity problem of unsupervised factor models.)

(c) Information leakage in unsupervised learning 3 marks

A common (and dangerous) reflex is to argue that, since PCA is unsupervised and there is no target, fitting it on the full dataset cannot leak information. Test this claim.

For both procedures below, use the same metric for "explained-variance ratio on Tuesday":

$$ev = 1 - \frac{\|X_{\text{tue}} - \tilde{X}_{\text{tue}}\|_F^2}{\|X_{\text{tue}}\|_F^2}$$

where X_{tue} is Tuesday's returns matrix centred by the procedure's training mean, and $\tilde{X}_{\text{tue}}$ is its reconstruction from the top- K eigenvectors of that procedure.

- **Correct procedure.** Fit PCA on Monday *only*. Centre Tuesday using Monday's mean (using a statistic computed from Tuesday itself is its own form of leakage). Project Tuesday's centred returns onto Monday's top- K eigenvectors and compute the metric above. Call this `ev_test_clean`.
- **Leaky procedure.** Fit PCA on Monday and Tuesday *combined*. Both the centring mean and the eigenvectors come from this combined fit. Project Tuesday onto these combined eigenvectors and compute Tuesday's explained-variance ratio. Call this `ev_test_leaky`.
- Quantify the leakage: `gap = ev_test_leaky - ev_test_clean`. Report the gap. The expected order of magnitude is small but non-zero (a few percentage points); a gap that is exactly zero or implausibly large suggests a bug in your procedure.
- Under what data-generating conditions would the gap be largest? Smallest? Relate your answer to the magnitude of the regime change between Monday and Tuesday.
- Translate the leakage gap into a practical consequence. The clean procedure's residual variance on Tuesday is the desk's honest estimate of the size of the unhedged risk after projecting onto the top- K factors. Does the leaky procedure over-state or under-state this out-of-sample residual variance, and in which direction does that bias the desk's perception of risk? Why does the fact that PCA has "no labels" not make it safe?

(d) Prove the neutrality claim from the geometry 2 marks

Without using simulation:

- Using **your** discovered PC1 and PC2 (not factors that were given to you), compute the loading of the equal-weight portfolio on PC1 and on PC2. The empirical numbers will be small but not exactly zero, due to sampling noise; that is expected.
- Now prove the *population* statement: assume the geometric arrangement you described in (b), namely that the population top-2 loading vectors place the 12 assets at the vertices of a regular 12-gon centred at the origin (the centring of the polygon is part of the assumption, not a consequence of return-centring; centring the returns places the score cloud at the origin, not the loadings cloud). Show algebraically that, under this assumption, the equal-weight portfolio's loadings on PC1 and PC2 are *exactly* zero. Your proof should be a few lines of algebra starting from $\sum_{k=0}^{11} e^{i \cdot 2\pi k/12} = 0$, not a numerical exercise.

(e) Observations 1 mark

Write 3 to 5 observations. At least one must connect your findings in (c) and (d). Specifically: the neutrality proof in (d) relied on a particular geometric arrangement of the loadings; leakage in (c) corrupts the loadings used as the input to that proof. Explain how the leaky procedure can break the polygon symmetry and therefore make the equal-weight loading non-zero, even though no "target" was ever leaked in the supervised sense.

Notebook skeleton (Part 1)

```
# Part 1: The Blind Cartographer
# All imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

#####
# (a) Load Monday ticks and discover the factor structure
#####
ticks_mon = pd.read_csv("ticks_mon.csv") # timestamp, asset_id, mid_return

# Pivot into a (T x 12) returns matrix aligned on timestamps
returns_mon = ticks_mon.pivot(index="timestamp", columns="asset_id", values="mid_return").dropna()

# Fit PCA
pca_mon = PCA().fit(returns_mon.values)

# Scree plot + Kaiser / broken-stick / parallel analysis
def select_n_components(eigenvalues, n_obs, n_vars, n_resamples=500, seed=0):
    # Return a dict: {"kaiser": k1, "broken_stick": k2, "parallel": k3}.
    # Implement all three. Each returns the number of "real" components.
    # The elbow rule from the prose is read off the scree plot by eye and is not part
    # of this dict; record your elbow verdict separately in a markdown cell.
    return verdicts

verdicts = select_n_components(pca_mon.explained_variance_, *returns_mon.shape)
print("Component-count verdicts:", verdicts)
# Plot the scree and annotate the cutoff suggested by each criterion.

#####
# (b) Plot loadings + rotation ambiguity
#####
loadings = pca_mon.components_[1:2].T # shape (12, 2)
# Scatter-plot loadings with asset_id annotations; describe the arrangement.

def rotate(loadings, angle_deg):
    theta = np.deg2rad(angle_deg)
    R = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
    return loadings @ R.T

# Plot rotated loadings; comment on what changed and what did not.

#####
```

```

# (c) Information leakage analysis
#####
ticks_tue = pd.read_csv("ticks_tue.csv")
returns_tue = ticks_tue.pivot(index="timestamp", columns="asset_id", values="mid_return").dropna()

def projected_explained_variance(test_X, eigenvectors_k, train_mean):
    # Centre test_X using train_mean (NOT test_X.mean()) to avoid leaking test-set statistics.
    # Project the centred test_X onto the first k eigenvectors and return the fraction of
    # test variance explained: 1 - ||residual||^2 / ||test_X_centered||^2
    return ev_ratio

K = verdicts["kaiser"] # or your chosen k from (a); if criteria disagree, pick one and justify

# Correct procedure: centre Tuesday using Monday's mean
train_mean = returns_mon.values.mean(axis=0)
ev_test_clean = projected_explained_variance(returns_tue.values, pca_mon.components_[0:K], train_mean)

# Leaky procedure
returns_combined = pd.concat([returns_mon, returns_tue]).values
pca_combined = PCA().fit(returns_combined)
combined_mean = returns_combined.mean(axis=0)
ev_test_leaky = projected_explained_variance(returns_tue.values, pca_combined.components_[0:K], combined_m

print("Clean ev_test :", ev_test_clean)
print("Leaky ev_test :", ev_test_leaky)
print("Leakage gap :", ev_test_leaky - ev_test_clean)

# Markdown cell: when is the gap largest? smallest? consequence for the desk?

#####
# (d) Structural proof of neutrality
#####
w_eq = np.ones(12) / 12
beta_PC1 = float(w_eq @ loadings[:, 0])
beta_PC2 = float(w_eq @ loadings[:, 1])
print(f"Equal-weight loading on PC1: {beta_PC1:.3e}")
print(f"Equal-weight loading on PC2: {beta_PC2:.3e}")

# In a markdown cell, prove from the geometry of (b) that these are exactly zero in the population.

#####
# (e) Observations
# 1.
# 2.
# 3.
#####

```

Part 2 • The Sentinel

At 13:00 on Tuesday the desk's risk monitoring system fired a soft alert. By 14:00 it was clear that something in the structure of the book had broken: pair-trade sub-books that were flat all morning suddenly began bleeding P&L. The headline volatility, however, looked normal, and no single asset moved by more than 2σ . **Eyeballing returns missed it.**

Your job is to build a **PCA-based real-time anomaly detector** that would have caught the event, and then show that it generalises by benchmarking it against alternative methods.

Background. See **Appendix A** for the definitions of the Q-statistic (Squared Prediction Error) and Hotelling's T^2 statistic. Both are standard tools in industrial process monitoring; this is likely the first time most candidates will have seen them applied to financial microstructure.

(a) Compute the PCA-based anomaly scores 2 marks

Pivot Tuesday's ticks into a $(T \times 12)$ returns matrix using the **same column order** as Monday's pivot, so that the asset axis aligns with the eigenvectors fit in Part 1. Drop any timestamps with missing values, exactly as in Part 1. **Throughout Part 2, “tick” means a post-drop tick (a row of the pivoted, NaN-dropped returns matrix), and the 13:00 cut-off is applied on the wall-clock timestamp of those surviving rows.** Centre Tuesday's returns using **Monday's column means** (do not recentre on Tuesday). Then, using the PCA model fit on Monday in Part 1 (top- k components, with k chosen in Part 1a), compute for *every* Tuesday timestamp:

- The **Q-statistic** (Squared Prediction Error): with r_t the centred return vector at time t and P the matrix whose rows are the top- k Monday eigenvectors, $Q_t = \|r_t - P^\top P r_t\|^2$.
- The **Hotelling's T^2 statistic**: $T_t^2 = \sum_{j=1}^k (r_t^\top v_j)^2 / \lambda_j$, with v_j the j -th Monday eigenvector and λ_j its training-set eigenvalue.

Plot both time series side by side. Shade the known event window 13:00 to 14:00 (inclusive) and comment on what you see.

(b) Detection thresholds and detection latency 2 marks

- Set a detection threshold for each statistic using Monday's data as the in-control baseline. Use both the empirical 99th percentile of Monday's series and a parametric control limit (Jackson-Mudholkar for Q, using Monday's *training* tail eigenvalues $\lambda_{k+1}, \dots, \lambda_{12}$ from the Part 1 fit, and the F-distribution for T^2 , both at significance $\alpha = 0.01$, with N_{train} equal to the number of Monday timestamps after the pivot and NaN drop). Report all four thresholds.
- **Detection latency.** For each statistic and each threshold, define detection latency as the first tick $t \geq 13:00$ at which the statistic is **strictly greater than** its threshold, measured in post-drop ticks (and seconds) after 13:00. If the statistic is already above the threshold at 13:00 because of an earlier morning crossing, this still counts as the detection time (latency = 0); the

cost of those morning crossings is captured separately by the false-positive rate, not by the latency.

- If the statistic does not cross its threshold anywhere on Tuesday, report the latency as "DNF" (did not fire) and exclude that row from any "earliest detector" comparison.
- Which statistic detects earlier, Q or T²? Explain in one paragraph why; in particular, what kind of disturbance does each statistic respond to (in-model vs out-of-model variation)?

(c) Contribution analysis: which asset broke? 2 marks

- Identify the peak-Q tick within the event window 13:00 to 14:00 (inclusive). Use the centring and projection from (a), so the residual at the peak tick is $r_t - P^\top P r_t$ with r_t centred by Monday's mean.
- Decompose Q into per-asset contributions: the contribution of asset i at tick t is the squared i -th component of the residual, $c_{t,i} = (r_t - P^\top P r_t)_i^2$. By construction, $\sum_i c_{t,i} = Q_t$.
- Plot the 12 contributions as a bar chart. Identify the top contributing asset(s); a reasonable rule is "any asset whose contribution exceeds a stated share of the total Q_t " (state the share you used, e.g. 10% or 20%).
- Do the top contributors form a coherent pair, a single outlier, or a diffuse pattern? What does this pattern suggest about the underlying cause of the event?

(d) Streaming detector with concept drift 2 marks

What this part is about. Parts 2(a) to (c) used a *static* PCA model fit once on Monday. In a live setting that model goes stale: volatilities drift, correlations breathe, and yesterday's "normal" is not today's "normal", a phenomenon known as **concept drift**. A streaming detector addresses this by maintaining a running, exponentially weighted estimate of the covariance and re-deriving the PCA from it on the fly. The trade-off you will observe is that a model which adapts to drift will also partially adapt to the very anomaly you are trying to detect, so an event that lasts long enough can be absorbed into the "new normal" and the Q-statistic will decay even while the desk is still bleeding. Picking λ is the knob that trades off adaptivity against detection power.

Build a streaming detector that processes Tuesday's ticks *one at a time*:

- Maintain an **exponentially weighted** covariance estimate $\hat{\Sigma}_t = (1 - \lambda) \hat{\Sigma}_{t-1} + \lambda r_t r_t^\top$ with $\lambda = 0.01$. Note the convention: here λ is the *innovation weight* (the multiplier on the new outer product), not the decay factor; equivalently the decay is $1 - \lambda = 0.99$. The recursion uses $r_t r_t^\top$ rather than $(r_t - \hat{\mu}_t)(r_t - \hat{\mu}_t)^\top$; for this case study you may assume intraday returns are sufficiently close to zero-mean that the uncentred recursion is acceptable. (You may instead maintain an EWMA mean and centre at each tick if you prefer; state your choice.) Because $\hat{\Sigma}_0 = 0$, the recursion is biased toward zero for small t (the effective weight on the first tick is λ , not unity); the warmup below absorbs this transient and no explicit bias

correction is required, but you may divide by $1 - (1 - \lambda)^t$ if you prefer an unbiased estimator (state your choice).

- The streaming detector starts on Tuesday's first tick from $\hat{\Sigma}_0 = 0$ and is *not* seeded with Monday's covariance; the warmup below absorbs the resulting transient. Use a **warmup** of (say) 500 ticks during which Q and T^2 are not emitted. State your warmup length.
- After warmup, re-derive the top- k eigenvectors at each tick (same k as in Part 1; you may use rank- k incremental updates, and full eigendecomposition of the 12×12 covariance every tick is acceptable if it meets the latency budget). Re-deriving on a fixed cadence (e.g. every M ticks) instead of every tick is also acceptable provided M is stated and justified, and the same eigenvectors are then re-used for Q and T^2 on the intervening ticks. Streaming eigenvectors flip sign and may re-order tick-to-tick; this does *not* affect Q_t (the projector $P^\top P$ is invariant to sign flips and to ordering within the top- k subspace). For T_t^2 , pair each $v_j(t)$ with the corresponding eigenvalue of $\hat{\Sigma}_t$, not with Monday's λ_j .
- Output Q and T^2 per tick relative to the rolling model.
- **Thresholds for the streaming detector:** for grading parity, use the same empirical 99th-percentile thresholds you computed from *Monday's static-PCA* Q and T^2 series in (b). You may, in addition, propose and discuss a tick-by-tick parametric limit, but the comparison plots and table below should use the static thresholds.
- Latency budget: aim for **50 μ s per tick** on a single core, and report your numbers honestly even if you miss it. Benchmark and report your mean and 99th-percentile per-tick latency. (Hint: use `time.perf_counter_ns()` around each `update()` call for nanosecond-resolution timing, then convert to μ s.)
- Plot the streaming Q against the static (Monday-only) Q from (a) overlaid on the same axes. Which detects the event sooner? Which raises more false positives in the morning? Explain the trade-off to a non-technical risk manager in two sentences.

(e) Model shootout (open-ended) 2 marks

PCA residuals are one of many ways to detect anomalies. You must implement and benchmark your PCA detector against **at least two** alternative methods. Suggested options (you may use any other method, but you must justify it):

- Isolation Forest on the raw 12-asset returns.
- One-Class SVM on the PCA-projected scores.
- Local Outlier Factor (LOF) on a rolling window.
- Autoencoder reconstruction error (a small dense or 1-D conv net is fine).
- Mahalanobis distance using a robust covariance estimator (MCD).

For each method, train on Monday's data and evaluate on Tuesday. Apply the following conventions for fair comparison across rows:

- If the alternative consumes PCA-projected scores (e.g. One-Class SVM on PC scores), use the **Monday static PCA** from Part 1 and project Tuesday using its eigenvectors and Monday's centring mean.
- **Train/threshold/evaluate split.** Fit the model itself on the **first 80%** of Monday's ticks. Set the detection threshold to the 99th percentile of the model's anomaly score on that *same* first 80% (in-sample). Compute the false-positive rate on the **last 20%** of Monday (held-out). Evaluate detection on Tuesday.
- **Anomaly-score sign convention.** Use a score where *higher means more anomalous*. For scikit-learn estimators whose `decision_function` / `score_samples` return higher-means-more-normal (Isolation Forest, One-Class SVM, LOF), negate the output before thresholding.
- **False-positive rate on Monday:** the fraction of held-out (last-20%) Monday ticks at which the score exceeds the threshold set on the first 80%.
- **Detection latency on Tuesday:** same definition as in (b); the first Tuesday tick at $t \geq 13:00$ at which the score is strictly greater than the method's own in-sample 99th-percentile threshold.
- **Mean latency per tick:** the wall-clock time of a single-row scoring call (`predict` or `decision_function` on a 1×12 array), averaged over all Tuesday ticks. Use `time.perf_counter_ns()`. Do not include training time.

Report a comparison table with the following columns. The “Static PCA (Q)” row uses the Part 1 PCA fit on the full Monday series (not refit on 80%) and the **empirical 99th-percentile** threshold from (b) (parity with the alternative methods, which all use empirical thresholds, and parity with how the desk would actually deploy the Part 1 model); its “false-positive rate on Monday” is therefore the share of the full Monday series above the (b) threshold, by construction approximately 1%. The alternative-method rows follow the 80/20 split defined above. Report the Jackson–Mudholkar latency separately in the (b) write-up.

Method	Detection latency (ticks after 13:00)	False-positive rate on Monday	Mean latency per tick (μ s)
Static PCA (Q)	...	...	...
Streaming PCA (Q)	...	...	...
Alternative #1	...	...	...
Alternative #2	...	...	...

Commit, in writing, to **one** method as your production recommendation. Your justification must connect ML performance metrics to the practical constraints of the desk: latency (microseconds matter), false-positive cost (each false alarm is one unnecessary liquidation), and interpretability (the risk manager will want to know "why" when it fires).

(f) Observations included above

Part 2 is marked out of 10: (a) 2 + (b) 2 + (c) 2 + (d) 2 + (e) 2 = 10. There is no separate allocation for (f); use this section for one short paragraph (or 3 to 5 bullets) capturing any lessons that did not fit into the (e) table or the production-recommendation paragraph. Do not duplicate content already written there.

Notebook skeleton (Part 2)

```
# Part 2: The Sentinel
# All imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
# Add other imports for your chosen alternatives

ticks_tue = pd.read_csv("ticks_tue.csv")
returns_tue = ticks_tue.pivot(index="timestamp", columns="asset_id", values="mid_return").dropna()

# Re-use pca_mon and K from Part 1
K = ... # your chosen number of components
eigvecs_k = pca_mon.components[:K] # (K, 12)
eigvals_k = pca_mon.explained_variance[:K] # (K,)

#####
# (a) Q and T-squared on Tuesday
#####
def q_statistic(X, eigvecs_k):
    #  $Q_t = || r_t - P^T P r_t ||^2$  where  $P = \text{eigvecs}_k$ 
    return q_series

def t2_statistic(X, eigvecs_k, eigvals_k):
    #  $T2_t = \sum_j (r_t \cdot v_j)^2 / \lambda_j$  for  $j = 1..K$ 
    return t2_series

q_tue = q_statistic(returns_tue.values, eigvecs_k)
t2_tue = t2_statistic(returns_tue.values, eigvecs_k, eigvals_k)

# Plot both with the 13:00-14:00 event window shaded

#####
# (b) Thresholds + detection latency
#####
def jackson_mudholkar_threshold(eigenvalues_tail, alpha=0.01):
    # Closed-form parametric control limit for Q (see Appendix A)
    return threshold

def t2_threshold_F(K, N_train, alpha=0.01):
    # F-distribution-based control limit for  $T^2$ 
```

```

    return threshold

# Compute empirical 99th-percentile thresholds from Monday Q and T^2 series too.
# Report all four thresholds and the corresponding first-crossing tick on Tuesday.

#####
# (c) Contribution analysis at the peak Q tick
#####
def per_asset_contributions(X, eigvecs_k):
    # Return an (T, 12) matrix of squared residuals per asset per tick
    return contrib

# Identify the peak Q tick in the event window, plot the 12 bars, interpret.

#####
# (d) Streaming detector with EWMA covariance
#####
class StreamingPCADetector:
    def __init__(self, n_assets, K, lam=0.01, warmup=500):
        self.K = K
        self.lam = lam
        self.warmup = warmup
        self.Sigma = np.zeros((n_assets, n_assets))
        self.t = 0

    def update(self, r):
        # 1. Update EWMA covariance
        # 2. If t >= warmup: eigendecompose, keep top-K eigvecs/eigvals
        # 3. Compute Q and T^2 for r against the current model
        # 4. Return (q, t2)
        return q, t2

# Run the detector tick-by-tick over Tuesday; time each update.
# Plot streaming-Q vs static-Q overlay.

#####
# (e) Model shootout
#####
def evaluate_method(name, train_X, test_X, scorer):
    # scorer is a callable that takes a row and returns an anomaly score
    # Return: detection_latency_ticks, fp_rate_on_train, mean_us_per_tick
    return row

# Implement at least two alternatives. Build the comparison table.
# Markdown cell: commit to one method for production with justification.

#####
# (f) Observations
# 1.
# 2.
# 3.
#####

```

Part 3 • The Regime Hunt

By Wednesday morning the loadings plot looks *almost* the same as Monday's. The PM is ready to flip the satellite strategies back on. You are not: pair-trade sub-books that were flat all of Monday lose money steadily across Wednesday and Thursday despite the headline picture being unchanged. Something, or several somethings, keep shifting under the hood.

This part is open-ended. You receive `ticks_wed.csv` and `ticks_thu.csv`. On both days each distinct timestamp belongs to exactly one of four regimes (regimes are episodic: a regime persists over a contiguous block of timestamps before switching):

Regime label	Description
NORMAL	The factor structure matches Monday's. Loadings stable, residuals in-control.
FAT_FINGER	Within a single episode, one asset (the same asset for the duration of that episode) is dislocated by a fixed offset in factor space. Eleven assets behave normally; one is mispriced. Different FAT_FINGER episodes may involve different assets and different offsets.
SECTOR_ROTATION	The eigenvalue spectrum changes (dispersion is squashed along one latent direction and stretched along another) while the top eigenvectors remain approximately aligned with Monday's. No single outlier; the shape of the loadings cloud has changed but identities are preserved.
CORRELATION_BREAK	The pairwise correlation structure has scrambled by a fixed permutation of asset identities for the duration of the episode (sector blocks are not preserved). Eigenvalues remain roughly Monday-like; eigenvectors do not.

Evaluators hold a hidden `labels_thu.csv` with the ground-truth regime at each Thursday timestamp. **Your Thursday predictions will be scored by macro-F1.**

Data availability and conventions

- **Monday raw ticks (`ticks_mon.csv`)** are provided alongside the fitted Part 2 PCA model and may be used to manufacture synthetic training examples (e.g. inject a fixed offset into one asset to produce FAT_FINGER samples). Monday is taken to be entirely NORMAL for any such use.
- **`labels_wed.csv` is released** alongside Wednesday's ticks so that Appendix B's `score_submission` code can compute F1 against ground truth on Wednesday.

labels_thu.csv is hidden.

- **Episode persistence.** Every regime episode lasts at least 30 consecutive ticks. Single-tick regime flips do not occur, so HMM-style temporal smoothing is appropriate.
- **Sector information.** No sector mapping is shipped; SECTOR_ROTATION must be detected from the data alone.
- **Warm-up.** A prediction is required for every Thursday timestamp, including those before your rolling window is full. Predict NORMAL for warm-up rows; missing rows in the submission will be treated as misclassifications.
- **Look-ahead detection.** Submissions are spot-checked by re-running the classifier on permuted suffixes of the Thursday stream and verifying that predictions up to time t are invariant to anything occurring after t . Submissions that change are zeroed.

(a) Feature engineering 2 marks

Design a feature set computed in a streaming fashion from the tick data. Your feature set **must include** all of the following, computed on a rolling window. You may use one shared window length across the four mandated features or a different length per feature; either is acceptable provided each is stated and justified. Calibrating against Monday's full-day PCA (eigenvalues, eigenvectors, residual model) is permitted and does not constitute look-ahead; only future ticks within the current day are forbidden.

- The **top-2 eigenvalue ratios** of the rolling covariance, defined as $\lambda_1 / \sum_{i=1}^{12} \lambda_i$ and $\lambda_2 / \sum_{i=1}^{12} \lambda_i$ where $\lambda_1 \geq \lambda_2 \geq \dots$ are the eigenvalues of the rolling covariance matrix (regime fingerprint).
- The mean pairwise correlation across all $\binom{12}{2} = 66$ asset pairs.
- The PCA reconstruction error (Q-statistic from Part 2) against the Monday model, using the same K chosen in Part 1(a).
- A **loading stability metric**: the principal-angle distance between the rolling top-2 eigenvectors and Monday's top-2 eigenvectors, reported in **degrees** in a column named subspace_angle_deg. The two mandated “top-2” features above are fixed at two components irrespective of the value of K chosen in Part 1(a). (Hint: use `scipy.linalg.subspace_angles` or implement it from SVD; convert from radians. Note that principal angles are sign-invariant; raw cosine similarity between eigenvectors is not, and will give spurious flips.)

You may add any additional features you wish. Document each in a markdown cell with one sentence explaining what regime signal it targets.

(b) Build a streaming regime classifier 3 marks

Build a model that maps the feature vector at each timestamp to one of the four regime labels. **Any method is allowed:**

- A rule-based decision tree on hand-thresholded features.
- An unsupervised clustering approach (KMeans, GMM, HDBSCAN) over the feature space, with cluster-to-label assignment by inspection.
- A change-point detector (e.g. ruptures) followed by per-segment classification.
- A Hidden Markov Model over the feature space.
- A supervised classifier trained on (i) synthetic regime examples manufactured from Monday ticks, (ii) the released Wednesday labels, or (iii) a self-labelled subset of Wednesday. Self-labelling or training on any portion of Thursday counts as look-ahead.

Hard constraints:

- Must operate in a streaming fashion: at time t you may use any features computed from data up to and including t , but **nothing from the future**. Look-ahead in any form will zero this part.
- Rolling window length is your choice but must be stated and justified. The window must cover at least 12 ticks (the dimension of the covariance) and a window noticeably shorter than the 30-tick minimum episode length will under-resolve regime boundaries; we suggest ≥ 60 ticks as a sensible floor, but you are free to choose otherwise with justification.
- Output a prediction for *every* Thursday timestamp (warm-up rows default to NORMAL as stated above).
- Predictions must be deterministic. If a probabilistic classifier returns near-equal scores, break ties by the fixed priority NORMAL > SECTOR_ROTATION > CORRELATION_BREAK > FAT_FINGER, and state the rule in your write-up.

(c) Self-evaluation on Wednesday 2 marks

- Independently of the released `labels_wed.csv` (i.e. without consulting it while you label), hand-label at least **5 distinct contiguous segments** of Wednesday's data (each represented by a representative timestamp and a stated start/end) by visual inspection of returns, loadings, correlations, or any plots you produce. The five segments must collectively cover at least three of the four regimes; five all-NORMAL segments will not score. For each, write 1 to 2 sentences explaining what you observed and why you assigned that label.
- Run your classifier on Wednesday and report per-regime precision, recall, and F1, plus macro-F1, against the released `labels_wed.csv`. Separately compare your hand-labels to the released labels and comment on any disagreements.
- A self-scoring script `score_submission` code is provided in **Appendix B**; it consumes `labels_wed.csv` as ground truth. Use it to validate your Wednesday output before submission. It is *not* provided for Thursday: Thursday is the hidden test set.

(d) Submission 2 marks

Produce `Team_{TeamNumber}_submission.csv` with columns `timestamp`, `predicted_regime` covering every *distinct* Thursday timestamp on the post-pivot, NaN-dropped tick grid (the same grid `labels_thu.csv` is indexed on; one row per timestamp, not one row per (timestamp, asset) pair). Timestamps must be written in ISO-8601 UTC format so that `pd.read_csv(..., parse_dates=["timestamp"])` in `score_submission` code recovers them exactly. Include the file in your submission zip.

Scoring (computed on the hidden Thursday ground truth; Wednesday is diagnostic only):

- Macro-F1 ≥ 0.85 : full 2 marks. Top 3 teams overall: **+2 bonus marks** (ties broken by earliest valid submission timestamp).
- Macro-F1 in $[0.65, 0.85)$: 1.5 marks.
- Macro-F1 in $[0.45, 0.65)$: 1 mark.
- Below 0.45 but methodology sound and write-up clear: 0.5 marks (no zero for honest effort).
- Look-ahead detected, malformed CSV, or no submission: 0 marks.

(e) Post-mortem and limitations 1 mark

In a markdown cell, answer:

- What is the minimum rolling-window length your detector needs to work reliably? How did you determine this?
- Which regime is hardest for your model to detect, and why?
- What would you change given one additional calibration week and one additional trading day of data?

Notebook skeleton (Part 3)

```
# Part 3: The Regime Hunt
# All imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.linalg import subspace_angles
# Plus whatever ML / clustering / HMM library you choose

ticks_wed = pd.read_csv("ticks_wed.csv")
ticks_thu = pd.read_csv("ticks_thu.csv")
returns_wed = ticks_wed.pivot(index="timestamp", columns="asset_id", values="mid_return").dropna()
returns_thu = ticks_thu.pivot(index="timestamp", columns="asset_id", values="mid_return").dropna()

REGIMES = ["NORMAL", "FAT_FINGER", "SECTOR_ROTATION", "CORRELATION_BREAK"]

#####
# (a) Streaming feature pipeline
#####
```

```

def streaming_features(returns_df, window, monday_eigvecs_k, monday_model):
    """
    Returns a DataFrame indexed by timestamp with at least:
        ev_ratio_pc1, ev_ratio_pc2, mean_corr, q_stat, subspace_angle_deg,
        ... plus any custom features you add.
    NO LOOK-AHEAD. Each row uses only data <= its timestamp.
    """
    return features_df

features_wed = streaming_features(returns_wed, window=300, monday_eigvecs_k=eigvecs_k, monday_model=pca_mon)
features_thu = streaming_features(returns_thu, window=300, monday_eigvecs_k=eigvecs_k, monday_model=pca_mon)

#####
# (b) Streaming classifier
#####
class RegimeClassifier:
    def fit(self, features_df, labels):
        # Optional. May also be hand-coded thresholds with no fit step.
        return self

    def predict_stream(self, features_df):
        # Walk row-by-row; return a Series of regime labels of the same length as features_df.
        # ALLOWED: rolling smoothing over past features.
        # FORBIDDEN: using any future row to compute the prediction at time t.
        return predictions

clf = RegimeClassifier().fit(...) # your training procedure
preds_wed = clf.predict_stream(features_wed)
preds_thu = clf.predict_stream(features_thu)

#####
# (c) Self-evaluation on Wednesday
#####
hand_labels = pd.DataFrame([
    # {"timestamp": ..., "label": "FAT_FINGER", "reason": "..."},
    # ...at least 5 rows...
])

# Compute precision / recall / F1 / macro-F1 against hand_labels.
# Use score_submission code (Appendix B) for sanity-check on Wednesday only.

#####
# (d) Build the Thursday submission CSV
#####
submission = pd.DataFrame({
    "timestamp": features_thu.index,
    "predicted_regime": preds_thu.values,
})
submission.to_csv("Team_{TeamNumber}_submission.csv", index=False)

#####
# (e) Post-mortem markdown cell
# - Minimum window length and how determined
# - Hardest regime

```

```
# - What you'd do with more data
#####
```

Part 4 • The Kill-Switch

Independent of the factor-structure question, the automated market-making engine accumulates inventory one fill at a time. Every fill nudges net inventory by $+1$ or -1 lot. The exchange severs the API connection the instant inventory reaches $+U = +40$ or $-L = -40$. The engine operates in *sessions*: each session targets $T = 10$ completed round-trip trades, after which the engine flattens, resets inventory to $i = 0$, and a new session begins. The trading day is composed of many such sessions; ruin probability is defined and reported **per session**. A **“round-trip” is an excursion away from $i = 0$ that ends when the inventory next returns to $i = 0$** . A round-trip is therefore counted at the tick on which inventory crosses or lands back on zero; the very next non-zero step starts the next round-trip.

From Monday's tick data you can measure that, because of a small but persistent latency disadvantage versus a competitor co-located in the same exchange, the engine suffers mild adverse selection: the probability p that the next fill pushes inventory *toward* the nearest limit slightly exceeds the probability $q = 1 - p$ that it pushes inventory back toward zero. By “toward the nearest limit” we mean: when current inventory $i > 0$, p is the probability of a $+1$ step; when $i < 0$, p is the probability of a -1 step; the parameter p is assumed homogeneous in i (the engine's adverse-selection edge does not depend on how close it is to a barrier). You estimate $p \approx 0.515$ on Monday.

(a) Closed-form ruin probability 2 marks

Model the inventory as a random walk on $\{-L, -L + 1, \dots, U - 1, U\}$ with ± 1 steps, starting at $i = 0$, and absorbing barriers at both ends. The step distribution is governed by p as defined above (toward the nearest barrier when $i \neq 0$); when $i = 0$ neither barrier is nearer, and you may take the step at $i = 0$ to be symmetric ($\Pr[+1] = \Pr[-1] = 0.5$). The walk is stopped at whichever event occurs first: absorption at $\pm U$, or the completion of $T = 10$ round-trips (in the sense defined above, i.e. T separate excursions away from 0 that return to 0). **Ruin** is the event that absorption occurs first; if the target is met first, the session ends without ruin and any subsequent inventory dynamics are irrelevant.

Derive the **closed-form** probability of ruin starting from $i = 0$. An acceptable closed form is any analytic expression (a finite sum or integral over the per-round-trip ruin probability and absorption-conditional hitting-time distribution is fine) that does *not* rely on Monte Carlo. Pure simulation will be accepted but will receive at most 1 of 2 marks.

(b) Sensitivity and intuition 2 marks

- Plot ruin probability as a function of p for $p \in [0.500, 0.530]$.
- Find p^* such that ruin probability equals 0.5. Report numerically; if p^* falls outside the plotted range, extend the search range and state the value.
- Explain, in two sentences to a non-technical risk manager, why the curve is so steep near $p = 0.5$.

(c) Re-estimate p on Tuesday and Wednesday 2 marks

Constructing the fill stream from tick data. The tick data contains `mid_return` per asset per timestamp, not explicit fills. To derive an inventory path, use the **canonical construction for grading parity**: at each post-drop tick, take the *sign of the net imbalance* across the 12 assets, i.e. $\text{sign}(\sum_{i=1}^{12} \text{sign}(r_{t,i}))$. A positive value is a +1 step, negative is -1, and zero-imbalance ticks are **skipped** (they do not contribute to n in any count or standard error). You may, in addition, report a sensitivity check using the sign of the PC1 score or the sign of the largest mover, but the canonical construction is what the grader will assume.

Hints to get started: once you have a fill stream, classify each step as “toward” or “away” from the nearest barrier by comparing its sign to the current inventory sign. **When current inventory is $i = 0$ neither barrier is nearer; that step is excluded from the toward/away counts.** Then $\hat{p} = n_{\text{toward}} / (n_{\text{toward}} + n_{\text{away}})$ with standard error $\sqrt{\hat{p}(1 - \hat{p})/n}$ under the i.i.d. Bernoulli assumption of the gambler's-ruin model (no HAC adjustment expected). As a sanity check, your Monday estimate should land near the 0.515 figure given in the problem statement.

- From Tuesday's and Wednesday's tick streams, estimate $\hat{p}$ separately for each day, with standard errors.
- Has the latency disadvantage worsened? Report the implied change in ruin probability.
- If $\hat{p}$ changes across regimes detected in Part 3, report $\hat{p}$ *per regime*, using the Part 3 classifier output indexed to match the post-NaN-drop tick index. Regimes with fewer than 50 contributing fills should be reported with a “n too small” flag rather than a tight standard error. Does adverse selection depend on regime?

(d) Anomaly-driven dynamic barriers 3 marks

Now connect Part 2 and Part 4. Instead of fixed ± 40 barriers, design **dynamic barriers** that tighten when the Part 2 anomaly detector is in alert.

- Define a policy: when the streaming Q-statistic from Part 2(d) exceeds the **empirical 99th-percentile threshold from Part 2(b)** (the same threshold used in the Part 2(e) shootout), the effective barriers contract from ± 40 to $\pm U_{\text{tight}}$ with $U_{\text{tight}} < 40$. Pick U_{tight} as a tunable parameter. During the streaming-Q warmup window (where Q is not emitted) use the default

± 40 barriers. When Q drops back below threshold, barriers revert to ± 40 on the next tick (no hysteresis).

- Simulate the inventory walk under two policies: (i) fixed ± 40 barriers, (ii) anomaly-driven dynamic barriers. Run two evaluations side by side:
 - **Empirical path.** Apply both policies to the actual Tuesday and Wednesday fill streams from (c). Treat each day's fill stream as a sequence of independent sessions: walk the stream from $i = 0$ until the session is either ruined or completes $T = 10$ round-trips, then reset to $i = 0$ and start the next session. Report on each day separately the empirical session-level ruin rate (fraction of sessions absorbed) and the mean number of round-trips completed per session.
 - **Synthetic ensemble.** Draw $N = 1000$ synthetic Bernoulli walks, each run until either ruin or completion of $T = 10$ round-trips, with step probability $\hat{p}$ from (c) (using Tuesday's $\hat{p}$ for the Tuesday ensemble and Wednesday's $\hat{p}$ for the Wednesday ensemble). For the dynamic policy, drive the barriers from the *empirical* Q -stream of that day (so the alert windows are held fixed across seeds; align by tick index, recycling Q if the synthetic walk is shorter than the empirical fill stream), and seed each synthetic walk explicitly with `np.random.default_rng(seed)` for $\text{seed} \in \{0, \dots, 999\}$.
- Report for both policies, on each day separately and over the synthetic ensemble: empirical ruin probability (fraction of the 1000 synthetic walks absorbed), mean number of round-trips completed, and the **trading-volume cost**, defined as round-trips completed under fixed barriers minus round-trips completed under dynamic barriers, averaged over matched seed pairs (same Bernoulli-walk seed, two policies).
- Plot the inventory paths for one representative scenario per policy, with the anomaly window shaded. Verify visually that the dynamic policy clamps inventory in time.

(e) Constant skew vs linear skew vs adaptive (open-ended) 1 mark

Propose a complementary **quote-skewing** rule: an adjustment to bid/ask quotes as a function of current inventory i , intended to push p back toward 0.5 as $|i|$ grows.

- Compare three policies on matched data: *constant skew* (a fixed offset applied whenever $|i| > i^*$), *linear skew* (offset proportional to i), and *adaptive skew* (offset proportional to *both* i and the Part 2 anomaly score, where “anomaly score” is the streaming Q from Part 2(d) divided by its Part 2(b) empirical 99th-percentile threshold, so the score is dimensionless and equals 1 at the alert boundary).
- **Surrendered spread** is the cumulative absolute skew offset applied per fill, summed over the day, expressed in basis points of mid-price (state your conversion if you work in price units instead).
- Assume the stylised elasticity $p(s) = 0.5 + (p_0 - 0.5)e^{-\kappa s}$, where s is the per-fill skew offset (bps), p_0 is the unmitigated adverse-selection probability for the prevailing day/regime (from

(c)), and $\kappa = 0.05$ per bp. State this assumption explicitly; if you prefer a different parametric form, state it and justify briefly.

- Plot ruin probability against surrendered spread for all three. Which dominates, and how does the answer depend on κ ?

(f) Observations folded into (e)

Part 4 is marked out of 10: (a) 2 + (b) 2 + (c) 2 + (d) 3 + (e) 1 = 10. There is no separate allocation for (f); use this section for one short paragraph (or 3 to 5 bullets) capturing cross-Part lessons that did not fit into the (e) write-up. In particular, did your Part 2 detector earn its keep by reducing ruin probability in (d)? Do not duplicate content already written elsewhere.

Notebook skeleton (Part 4)

```
# Part 4: The Kill-Switch
# All imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

U = 40
L = 40
T = 10
P_MON_EST = 0.515

#####
# (a) Closed-form ruin probability
#####
def ruin_probability(p, U=U, L=L, T=T, start=0):
    # Closed form preferred. Show derivation in a markdown cell.
    # Standard gambler's ruin with two absorbing barriers (modified for round-trip target T).
    return prob

print("Ruin probability at p=0.515:", ruin_probability(0.515))

#####
# (b) Sensitivity
#####
ps = np.linspace(0.500, 0.530, 61)
ruin = [ruin_probability(p) for p in ps]
plt.plot(ps, ruin); plt.xlabel("p"); plt.ylabel("Ruin probability")
plt.title("Ruin probability vs adverse-selection probability"); plt.grid(True); plt.show()
# Find p* with ruin == 0.5. Two-sentence intuition for the steepness in a markdown cell.

#####
# (c) Re-estimate p on Tuesday and Wednesday (with per-regime breakdown)
#####
def estimate_adverse_p(ticks_df, regime_series=None):
    # Step 1: Construct fill stream from tick data.
    # One approach: at each timestamp, sum the signs of mid_return across the 12 assets.
```

```

# fill_direction = +1 if sum > 0 (net buy), -1 if sum < 0 (net sell), skip if 0.
# Step 2: Walk the fill stream to build an inventory path starting at 0.
# Step 3: At each step, check if the fill pushed inventory toward or away from
# the nearest barrier. p_hat = (#toward) / (#toward + #away).
# Step 4: Standard error: se = sqrt(p_hat * (1-p_hat) / n_fills).
# If regime_series is provided, return a dict {regime: (p_hat, se)}.
return result

p_tue = estimate_adverse_p(ticks_tue)
p_wed_per_regime = estimate_adverse_p(ticks_wed, regime_series=preds_wed) # from Part 3
print("Tue:", p_tue, "ruin:", ruin_probability(p_tue[0]))
print("Wed per regime:", p_wed_per_regime)

#####
# (d) Anomaly-driven dynamic barriers (connects to Part 2)
#####
def simulate_inventory(fill_stream, barrier_fn, T=T):
    # barrier_fn(t) -> current (lower, upper) barriers at time t.
    # Walk the fill stream. Return: was_absorbed, n_round_trips_completed, inventory_path.
    return outcome

def fixed_barrier_fn(t):
    return (-L, U)

def anomaly_driven_barrier_fn(q_stream, q_threshold, U_tight):
    def fn(t):
        return (-U_tight, U_tight) if q_stream[t] > q_threshold else (-L, U)
    return fn

# Run many seeds and compare empirical ruin + cost.

#####
# (e) Skew policy shootout
#####
def skew_policy_constant(i, base_spread, sigma): ...
def skew_policy_linear(i, base_spread, beta): ...
def skew_policy_adaptive(i, q_t, base_spread, beta, gamma): ...

# Plot ruin vs surrendered spread for all three policies under matched conditions.

#####
# (f) Observations (connect to Parts 1-3)
# 1.
# 2.
# 3.
#####

```

Bonus • The Full Stack OPTIONAL • +5 MARKS

Build a single end-to-end pipeline that strings together everything you built in Parts 1 to 4 and runs it on a simulated one-hour feed. The bonus rewards teams who treat the case study as *one system* rather than four disconnected exercises.

Requirements

Your pipeline must implement the following components, in a single Python class or module that can process a stream of tick batches. The pipeline operates on the same post-pivot, NaN-dropped 12-vector grid used in Parts 1 to 4: each call to `step()` takes one row of the pivoted returns matrix (a 12-vector aligned with Monday's column order), not a single long-form (`timestamp`, `asset_id`, `mid_return`) tuple. Pivoting and the any-asset-NaN drop are the pipeline caller's responsibility and are illustrated in the demo cell below.

1. **Factor discovery** (from Part 1): on startup, fit PCA on **Monday's returns matrix** (the canonical calibration day) and persist the eigenvectors/eigenvalues.
2. **Streaming anomaly detection** (from Part 2d): maintain the rolling EWMA-covariance Q and T^2 statistics from Part 2(d) and emit an ALERT event on the rising edge only (i.e. on the first tick at which Q or T^2 crosses its threshold after a stretch of being below; do not log an event on every alert tick). Use the **empirical 99th-percentile thresholds from Part 2(b)** passed in via the `q_threshold` and `t2_threshold` constructor arguments; do not invent new thresholds inside the pipeline.
3. **Regime classification** (from Part 3): emit a regime label per tick using your Part 3 classifier. During the classifier's warmup window default to NORMAL.
4. **Kill-switch with dynamic barriers** (from Part 4d): drive the inventory simulator's barriers from the current anomaly state, using the canonical fill construction from Part 4(c) (sign of net imbalance across the 12 assets); halt on absorption. After absorption, subsequent `step()` calls return the absorbed snapshot unchanged, and a single ABSORBED event is recorded in telemetry.
5. **Telemetry**: at the end of the run, emit a JSON-line log of: alerts raised (rising-edge events as defined above), regime transitions (logged only when the label changes from the previous tick), **inventory peaks** (the running maximum of $|i|$ over the run, logged once at the end with the timestamp at which the maximum was first attained), ruin/no-ruin verdict, and total round-trips completed.

Demo protocol

Provide a single cell at the end of your notebook titled `### LIVE DEMO ###` that streams Thursday's first 60 minutes of post-pivot, NaN-dropped ticks through the pipeline in chunks of 100 ticks and prints the telemetry log as it goes. The 60-minute window is taken on the pivoted grid as $\{t : t < t_0 + 1 \text{ hour}\}$ where t_0 is the first post-drop Thursday timestamp (right-exclusive, as in the skeleton). The streaming detector cold-starts at the first Thursday tick (its EWMA warmup absorbs

the transient); Monday's PCA calibration is loaded once at construction time. The demo must complete in under 60 seconds wall-clock; the brief `time.sleep` in the skeleton is illustrative only and the 60-second cap takes precedence.

Marking

Criterion	Marks
Pipeline runs end-to-end without manual intervention	1
All four components correctly wired (data flows from one to the next)	1
Anomaly detector drives the kill-switch (not just decoration)	1
Telemetry log is structured, complete, and human-readable	1
Code quality: clean interfaces, no mutable global state, reasonable error handling	1

Skeleton (Bonus)

```
# Bonus: The Full Stack
# All imports
import json, time
import numpy as np
import pandas as pd

class PhantomLatticePipeline:
    """End-to-end live trading risk system. Wires Parts 1-4 together."""

    def __init__(self, calibration_returns, K, q_threshold, t2_threshold,
                 U=40, L=40, U_tight=20, target_round_trips=10):
        # 1. Fit PCA on calibration_returns -> store eigvecs_k, eigvals_k
        # 2. Initialise StreamingPCADetector (from Part 2d)
        # 3. Initialise RegimeClassifier (from Part 3); warm up if needed
        # 4. Initialise inventory state and dynamic barrier policy (from Part 4d)
        # 5. Initialise telemetry log (list of JSON-serialisable events)
        ...

    def step(self, tick_row):
        """Process ONE tick. Return a dict with current alert/regime/inventory state."""
        # 1. Update streaming PCA -> (q, t2)
        # 2. If q or t2 > threshold: log ALERT event
        # 3. Update streaming regime features -> regime_label; log on change
        # 4. Apply fill from tick to inventory under current barriers; check absorption
        # 5. Return current snapshot
        return snapshot

    def emit_telemetry(self):
        """Return the JSON-line log as a list of dicts."""
        return self.telemetry
```

```

### LIVE DEMO ###
pipe = PhantomLatticePipeline(
    calibration_returns=returns_mon.values,
    K=K, q_threshold=..., t2_threshold=...,
)

# Pivot Thursday's long-form ticks into the (T x 12) grid the pipeline expects.
# Use the SAME column order as Monday so step() vectors align with the calibration eigenvectors.
returns_thu = (ticks_thu
    .pivot(index="timestamp", columns="asset_id", values="mid_return")
    .dropna()
    .reindex(columns=returns_mon.columns))

t0 = returns_thu.index.min()
thu_first_hour = returns_thu.loc[returns_thu.index < t0 + pd.Timedelta("1H")]
chunks = np.array_split(thu_first_hour.values, max(1, len(thu_first_hour) // 100))

for chunk in chunks:
    for r in chunk:          # r is a 12-vector (np.ndarray, shape (12,))
        snapshot = pipe.step(r)
        print(snapshot)
        time.sleep(0.05)    # simulate real-time pacing

print(json.dumps(pipe.emit_telemetry(), indent=2, default=str))

```

Appendix A • Q-statistic and Hotelling's T² Primer

Multivariate process monitoring uses two complementary statistics, both computed from a fitted PCA model with K retained components.

Notation

Let $r_t \in \mathbb{R}^{12}$ be the **Monday-mean-centred** return vector at time t (the same centring used throughout Part 2). Let $P \in \mathbb{R}^{K \times 12}$ stack the top- K eigenvectors as rows, and let $\lambda_1, \dots, \lambda_K$ be the corresponding eigenvalues. Throughout this appendix K denotes the number of retained components; this is the same quantity called k in the body of Part 2.

Q-statistic (Squared Prediction Error, SPE)

Measures variation *outside* the model, i.e. residual energy after projecting onto the top- K subspace:

$$Q_t = \|r_t - P^\top P r_t\|^2$$

Jackson-Mudholkar control limit at significance α :

$$Q_\alpha = \theta_1 \left[\frac{z_\alpha \sqrt{2\theta_2 h_0^2}}{\theta_1} + 1 + \frac{\theta_2 h_0 (h_0 - 1)}{\theta_1^2} \right]^{1/h_0}$$

where $\theta_i = \sum_{j=K+1}^{12} \lambda_j^i$ for $i = 1, 2, 3$, and $h_0 = 1 - \frac{2\theta_1\theta_3}{3\theta_2^2}$, and z_α is the **upper-tail standard-normal quantile**, $z_\alpha = \Phi^{-1}(1 - \alpha)$ (positive for $\alpha < 0.5$; e.g. $z_{0.01} \approx 2.326$).

Hotelling's T² statistic

Measures variation *within* the model, i.e. how unusually far the projected point sits inside the top- K subspace, scaled by the in-sample variance along each PC:

$$T_t^2 = \sum_{j=1}^K \frac{(r_t^\top v_j)^2}{\lambda_j}$$

F-distribution control limit at significance α for N training observations, with $F_\alpha(K, N - K)$ denoting the **upper-tail F-quantile** ($\Pr[F > F_\alpha] = \alpha$):

$$T_\alpha^2 = \frac{K(N-1)(N+1)}{N(N-K)} F_\alpha(K, N-K)$$

Why both?

- **Q fires** when the data develops a new direction of variation that the training PCA did not model, e.g. a new factor, a regime change, or a structural break.
- **T² fires** when the data moves to an extreme location *along an existing modelled direction*, e.g. an unusually large move on PC1.
- An event that breaks the factor structure (like Tuesday's 13:00 incident) typically shows up on Q first, because the new variation is orthogonal to the trained subspace. A purely directional shock (a big market move along PC1) shows up on T² first.

Per-asset contributions to Q

The Q-statistic is additive across coordinates:

$$Q_t = \sum_{i=1}^{12} (r_t - P^\top P r_t)_i^2 = \sum_{i=1}^{12} c_{t,i}$$

Each $c_{t,i}$ is the contribution of asset i to the residual at time t . Used in Part 2(c) to identify which asset(s) broke.

Appendix B · Self-Scoring Script for Part 3

A script `score_submission` code is provided alongside the data files. It computes macro-F1 and per-regime F1 for your Wednesday predictions against the released Wednesday answer key `labels_wed.csv`. **It is not provided for Thursday**; Thursday is the hidden test set, evaluated by the case study panel. Per the rules in Part 3(d), any Thursday timestamp missing from your submission is treated as a misclassification.

The Wednesday predictions you feed to the script are a **local diagnostic file**; we suggest the name `Team_{TeamNumber}_wed_diagnostic.csv` below to keep it distinct from the Thursday submission. This file is for your own use during development and **must not be included in the submission zip**; only `Team_{TeamNumber}_submission.csv` (Thursday) is graded.

Run it from a notebook cell as below.

```
# Appendix B: self-scoring (Wednesday only)
import subprocess, json

result = subprocess.run(
    ["python", "score_submission code",
     "--predictions", "Team_{TeamNumber}_wed_diagnostic.csv",
     "--truth", "labels_wed.csv"],
    capture_output=True, text=True,
)
print(result.stdout)
scores = json.loads(result.stdout.splitlines()[-1])
print("Macro-F1:", scores["macro_f1"])
print("Per regime:", scores["per_regime_f1"])
```

Script source (also shipped as `score_submission` code):

```
#!/usr/bin/env python
# score_submission code: Part 3 self-scoring tool
import argparse, json
import pandas as pd
from sklearn.metrics import f1_score, precision_recall_fscore_support

REGIMES = ["NORMAL", "FAT_FINGER", "SECTOR_ROTATION", "CORRELATION_BREAK"]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--predictions", required=True, help="CSV with columns: timestamp, predicted_regime")
    ap.add_argument("--truth", required=True, help="Parquet with columns: timestamp, regime")
    args = ap.parse_args()
    preds = pd.read_csv(args.predictions, parse_dates=["timestamp"])
    truth = pd.read_csv(args.truth)

    # Validate format
```

```

assert set(preds.columns) == {"timestamp", "predicted_regime"}, "Bad prediction CSV columns"
invalid = set(preds["predicted_regime"]) - set(REGIMES)
assert not invalid, f"Invalid regime labels: {invalid}"

# Inner join on timestamp
merged = truth.merge(preds, on="timestamp", how="inner")
coverage = len(merged) / len(truth)
if coverage < 0.99:
    print(f"WARNING: coverage only {coverage:.1%}; missing predictions for some timestamps.")

y_true = merged["regime"].values
y_pred = merged["predicted_regime"].values

macro = f1_score(y_true, y_pred, labels=REGIMES, average="macro", zero_division=0)
p, r, f, _ = precision_recall_fscore_support(y_true, y_pred, labels=REGIMES, zero_division=0)

out = {
    "macro_f1": float(macro),
    "coverage": float(coverage),
    "per_regime_f1": dict(zip(REGIMES, [float(x) for x in f])),
    "per_regime_precision": dict(zip(REGIMES, [float(x) for x in p])),
    "per_regime_recall": dict(zip(REGIMES, [float(x) for x in r])),
}
print(json.dumps(out))

main()

```

Reminder. Do *not* attempt to use this script as a training oracle by querying it with crafted predictions to back out the labels. Per Part 3(d), your Thursday submission will be inspected for evidence of label leakage; suspicious submissions will be disqualified.

Submission Requirements

Submit the following files in your Problem_Statement_2 folder:

1. Team_{TeamNumber}.ipynb: the notebook covering Parts 1 to 4 (and the Bonus, if attempted). Must run end-to-end via "Run All" against the provided data files.
2. Team_{TeamNumber}_submission.csv: your Thursday regime predictions (Part 3d).
3. Any supporting data files you generated (intermediate features, model artefacts, etc.).

Scoring summary:

- Part 1 (10) + Part 2 (10) + Part 3 (10) + Part 4 (10) = **40 marks**.
- Part 3 leaderboard top-3 bonus: **+2 marks**.
- Bonus full-stack pipeline: **+5 marks**.
- Maximum achievable: **47 marks**.

